

CSE5095 — Homework 2 Walkthrough

Type Safety, by Worked Analogy

How to use this. Every section below works a *different* construct than the homework, but uses *the same technique* you need. Read the worked example, then do the matching task yourself (the fill-in-the-blank skeletons in `hw2_hints.tex` pair well with this). Nothing here is an answer to a homework task. Your syllabus allows AI for understanding the material; the submitted write-up has to be yours.

The four judgments you’ll use.

- $e \text{ val}$: e is finished evaluating.
- $e \mapsto e'$: e takes one step to e' .
- $e : \tau$: e has type τ .
- Theorems: **Progress** (if $e : \tau$ then $e \text{ val}$ or $\exists e'. e \mapsto e'$) and **Preservation** (if $e : \tau$ and $e \mapsto e'$ then $e' : \tau$).

The one distinction to keep in your head.

- A *search rule* has a *step* judgment as a premise (e.g. StepSearchAddLeft: “if $e_1 \mapsto e'_1$ then $e_1 + e_2 \mapsto e'_1 + e_2$ ”). It just says *where* to evaluate next.
- A *primitive / reduction rule* (StepAdd, StepCat, ...) does the actual computation. It has no step premises, though it may have side conditions like $n_2 \neq 0$.

In proofs, reduction-rule cases use **inversion**. Search-rule cases use **the induction hypothesis**.

Part A: Booleans (Tasks 1–6)

Worked construct: pairs. We extend the class language E with

$$\tau ::= \dots \mid \tau_1 \times \tau_2 \qquad e ::= \dots \mid (e_1, e_2) \mid \text{fst } e$$

Intended meaning: evaluate both components left to right; `fst` of a finished pair returns the first component.

A1. Writing dynamic rules

(technique for Task 1)

Recipe.

1. *Value rules:* which finished forms of the new syntax are done?
2. *Reduction rules:* once the sub-expressions you care about are values, what does the construct compute to?
3. *Search rules:* for each sub-expression that must be evaluated *before* reducing, add one rule that steps it and leaves everything else alone. Do **not** add search rules for sub-expressions that shouldn't be evaluated yet.

Worked. A pair is a value when both parts are values. This value rule *has premises*, unlike ValNum:

$$\frac{e_1 \text{ val} \quad e_2 \text{ val}}{(e_1, e_2) \text{ val}} \text{ (VALPAIR)}$$

Reduction: `fst` of a finished pair gives the first component:

$$\frac{e_1 \text{ val} \quad e_2 \text{ val}}{\text{fst } (e_1, e_2) \mapsto e_1} \text{ (STEPFST)}$$

Search: we need to evaluate the left component, then the right one (only once the left is a value), and the argument of `fst` :

$$\frac{e_1 \mapsto e'_1}{(e_1, e_2) \mapsto (e'_1, e_2)} \text{ (STEPSEARCHPAIRL)} \quad \frac{e_1 \text{ val} \quad e_2 \mapsto e'_2}{(e_1, e_2) \mapsto (e_1, e'_2)} \text{ (STEPSEARCHPAIRR)}$$
$$\frac{e \mapsto e'}{\text{fst } e \mapsto \text{fst } e'} \text{ (STEPSEARCHFST)}$$

Sanity checks to run on your own rules.

- For every non-value expression, is there *exactly one* rule that applies? (No rule gives a stuck expression; two rules means your semantics isn't deterministic.)
- Does anything get evaluated that the English description says *shouldn't* be? For if, reread the assignment's if `x = 0 then 0 else 42/x` remark, and ask which sub-expressions deserve a search rule.

Your turn: Task 1. Apply the recipe to `true`, `false`, and `if`. Target count from the assignment: 2 value rules, 3 step rules, exactly 1 of which is a search rule.

A2. Evaluating step by step

(technique for Task 2)

Recipe. At each line, find the *innermost-leftmost* piece that a reduction rule can fire on. Then name the stack of search rules that “reaches in” to it, from the outside in.

Worked. Evaluate $\text{fst } (\bar{1} + \bar{2}, \text{"a"}^{\wedge}\text{"b"})$:

$$\begin{array}{ll}
& \text{fst } (\bar{1} + \bar{2}, \text{"a"}^{\wedge}\text{"b"}) \\
\mapsto & \text{fst } (\bar{3}, \text{"a"}^{\wedge}\text{"b"}) \quad \text{StepSearchFst} \leftarrow \text{StepSearchPairL} \leftarrow \text{StepAdd} \\
\mapsto & \text{fst } (\bar{3}, \text{"ab"}) \quad \text{StepSearchFst} \leftarrow \text{StepSearchPairR} \leftarrow \text{StepCat} \\
\mapsto & \bar{3} \quad \text{StepFst}
\end{array}$$

$\bar{3}$ is a value by ValNum, so we stop. The pair $(\bar{3}, \text{"ab"})$ wasn't the end, because it sits inside fst .

The " $\leftarrow$ " stack is shorthand for a real derivation tree. Here is the first step written out in full. Put one like this in your write-up if you want to be extra rigorous:

$$\frac{\frac{\overline{\bar{1} + \bar{2} \mapsto \bar{3}} \text{ (STEPADD)}}{\overline{(\bar{1} + \bar{2}, \text{"a"}^{\wedge}\text{"b"}) \mapsto (\bar{3}, \text{"a"}^{\wedge}\text{"b"})} \text{ (STEPSEARCHPAIRL)}}{\overline{\text{fst } (\bar{1} + \bar{2}, \text{"a"}^{\wedge}\text{"b"}) \mapsto \text{fst } (\bar{3}, \text{"a"}^{\wedge}\text{"b"})} \text{ (STEPSEARCHFST)}}$$

The second step also needs a leaf $\bar{3}$ val (ValNum), because StepSearchPairR has that premise.

Your turn: Task 2. Your expression has three nesting levels: if $\supset = \supset +$. Your first step's stack has one rule per level.

A3. Typing derivations

(technique for Task 3)

Typing rules for pairs:

$$\frac{e_1 : \tau_1 \quad e_2 : \tau_2}{(e_1, e_2) : \tau_1 \times \tau_2} \text{ (TYPEPAIR)} \qquad \frac{e : \tau_1 \times \tau_2}{\text{fst } e : \tau_1} \text{ (TYPEFST)}$$

Recipe. Write the goal at the bottom. Look at the *outermost* syntax to pick the rule, write its premises above the line, and repeat until every branch ends in an axiom (a rule with no premises).

Worked. $\text{fst } (\bar{1} + \bar{2}, \text{"hi"}) : \text{int}$

$$\frac{\frac{\overline{\bar{1} : \text{int}} \text{ (TN)} \quad \overline{\bar{2} : \text{int}} \text{ (TN)}}{\bar{1} + \bar{2} : \text{int}} \text{ (TA)} \quad \overline{\text{"hi"} : \text{string}} \text{ (TS)}}{\overline{(\bar{1} + \bar{2}, \text{"hi"}) : \text{int} \times \text{string}} \text{ (TPAIR)}} \text{ (TFST)}$$

When a rule like TypeFst lets you choose (here, any τ_2), the sub-derivation decides it. With if, TypeIf forces both branches to have the same τ , and τ is also the type of the whole expression.

Your turn: Task 3. The outermost rule is TypeIf with *three* premises. The condition's sub-tree looks just like the $\bar{1} + \bar{2}$ sub-tree above, with one more rule (TypeEq) underneath.

A4. Preservation cases

(technique for Task 4)

Structure: induction on the derivation of $e \mapsto e'$, one case per step rule. Each case follows this template:

1. “Then $e = \dots$ and $e' = \dots$ ” (read off the rule).
2. Invert the typing rule for e ’s outermost form, to learn the types of its pieces.
3. *Reduction rule:* e' is one of those pieces (or a new literal), so its type is already known, or comes from an axiom.
Search rule: apply the IH to the premise $e_i \mapsto e'_i$, then re-apply the same typing rule.

Worked: Case StepFst. Then $e = \text{fst } (e_1, e_2)$ and $e' = e_1$, with e_1 **val** and e_2 **val**. We know $e : \tau$. The only rule concluding $\text{fst} \cdot : \tau$ is TypeFst, so by inversion $(e_1, e_2) : \tau \times \tau_2$ for some τ_2 . The only rule concluding a pair’s type is TypePair, so inverting again gives $e_1 : \tau$ and $e_2 : \tau_2$. So $e' = e_1 : \tau$. $\square$

Worked: Case StepSearchFst. Then $e = \text{fst } e_0$ and $e' = \text{fst } e'_0$ with $e_0 \mapsto e'_0$. By inversion on TypeFst, $e_0 : \tau \times \tau_2$ for some τ_2 . By the induction hypothesis on $e_0 \mapsto e'_0$, $e'_0 : \tau \times \tau_2$. By TypeFst, $e' = \text{fst } e'_0 : \tau$. $\square$

Watch out. In the search case, the IH is about the *sub-expression*’s type ($\tau \times \tau_2$ here), not τ . Say which type you’re applying it at.

Your turn: Task 4. Three cases, one per Task 1 step rule. Your two reduction cases each need *one* inversion (on TypeIf).

A5. Canonical Forms

(technique for Task 5)

Recipe. Canonical forms answers: “which value rules could possibly have produced a value of this type?” List those shapes.

Worked. If e **val** and $e : \tau_1 \times \tau_2$, then $e = (e_1, e_2)$ for some e_1, e_2 with e_1 **val** and e_2 **val**. (ValNum and ValString produce things of type **int** and **string**, so only ValPair can produce a value of a product type.)

Your turn: Task 5. Which of your Task 1 value rules produce something of type **bool**? A type may have more than one canonical form, and that’s fine. It just means a case split later.

A6. Progress cases

(technique for Task 6)

Structure: induction on the derivation of $e : \tau$. Template for a rule with sub-expressions:

1. Invert to get the sub-expression typings (the premises of the rule).
2. Apply the IH to the sub-expression that gets evaluated first. You get two cases: it’s a value, or it steps.

3. *Steps*: use the search rule. *Value*: use Canonical Forms to get its exact shape, then use the reduction rule for that shape.

Worked: Case TypeFst. Then $e = \text{fst } e_0$ with $e_0 : \tau \times \tau_2$. By the IH, e_0 **val** or $e_0 \mapsto e'_0$.

- Suppose e_0 **val**. By Canonical Forms (product case), $e_0 = (e_1, e_2)$ with e_1 **val** and e_2 **val**. By StepFst, $e \mapsto e_1$.
- Suppose $e_0 \mapsto e'_0$. By StepSearchFst, $e \mapsto \text{fst } e'_0$.

□

Worked: Case TypePair (for the value-rule side). Here *both* components get evaluated, so we need the IH on both, nested exactly like the class's TypeAdd case. If both are values, e **val** by ValPair (not a step!). If e_1 is a value and e_2 steps, use StepSearchPairR. If e_1 steps, use StepSearchPairL.

Your turn: Task 6. TypeTrue and TypeFalse are one-liners (like TypeNum). For TypeIf, how many sub-expressions does your semantics evaluate first? That tells you how many IHs you need. In the value case, Canonical Forms gives you *two* shapes, so you get two sub-cases.

Part B: Division by zero (Tasks 7–9)

Worked system: SQ, a language that prevents *square roots of negatives*. Types $\tau ::= \text{nn} \mid \text{any}$ (“definitely non-negative” vs. “any integer”). Expressions: $e ::= \bar{n} \mid e + e \mid e - e \mid e * e \mid \text{sqrt } e$, with arithmetic dynamics like EZ’s, plus

$$\frac{n \geq 0}{\text{sqrt } \bar{n} \mapsto \lfloor \sqrt{n} \rfloor} \text{ (STEPSQRT)} \quad \frac{e \mapsto e'}{\text{sqrt } e \mapsto \text{sqrt } e'} \text{ (STEPSEARCHSQRT)}$$

So $\text{sqrt } \overline{-4}$ is stuck, just like $\bar{1}/\bar{0}$ in EZ. As in EZ, if $e : \text{nn}$ then $e : \text{any}$, and Canonical Forms says nn-values are $\bar{n}$ with $n \geq 0$.

B1. Designing refinement rules (technique for Task 7)

Recipe (the whole trick). For each blank, try the *stronger* type first.

- To show the stronger type is **unsafe**, find *one* counterexample: inputs satisfying the premises where the output breaks the property.
- To show it’s **safe**, give an arithmetic argument that works for *all* such inputs.

Worked. Base rules: $\frac{}{\bar{n} : \text{any}} \text{ (TYPENUM)}$ and $\frac{n \geq 0}{\bar{n} : \text{nn}} \text{ (TYPENUMNN)}$.

- $e_1 : \text{nn}, e_2 : \text{nn} \Rightarrow e_1 + e_2 : ?$ A sum of two non-negatives is ≥ 0 for *all* inputs. Safe, so nn.
- $e_1 : \text{nn}, e_2 : \text{nn} \Rightarrow e_1 - e_2 : ?$ Counterexample: $0 - 1 = -1$. So only any.
- $e_1 : \text{nn}, e_2 : \text{nn} \Rightarrow e_1 * e_2 : ?$ A product of non-negatives is ≥ 0 . So nn.
- $\text{sqrt } e$: StepSqrt’s side condition is $n \geq 0$, so the premise must be $e : \text{nn}$. Result: $\lfloor \sqrt{n} \rfloor \geq 0$ always. So nn.

Plus any-versions ($e_1 : \text{any}, e_2 : \text{any} \Rightarrow e_1 \text{ op } e_2 : \text{any}$) so that mixed programs still type check.

Your turn: Task 7. Same drill for EZ’s blanks. The assignment reminds you integers can be negative, and division truncates. Test $\bar{1}/\bar{2}$ before you decide the result type of TypeDiv.

B2. Progress with a side condition (technique for Task 8, Progress)

Worked: Case TypeSqrt. Then $e = \text{sqrt } e_0$, $\tau = \text{nn}$, and $e_0 : \text{nn}$. By the IH, $e_0 \text{ val}$ or $e_0 \mapsto e'_0$.

- Suppose $e_0 \text{ val}$. By Canonical Forms (**the nn case**), $e_0 = \bar{n}$ with $n \geq 0$. That’s exactly StepSqrt’s side condition, so $e \mapsto \lfloor \sqrt{n} \rfloor$.
- Suppose $e_0 \mapsto e'_0$. By StepSearchSqrt, $e \mapsto \text{sqrt } e'_0$.

□

The point: the *type* of the argument is what gives you the side condition, through Canonical Forms. That’s the whole reason the type system exists. TypeDiv has two sub-expressions, so you’ll nest two IHs (like the class’s TypeAdd case) and use the nz case of Canonical Forms only on the one where it matters.

B3. Preservation when inversion gives two rules (Task 8 & Bonus)

Worked: Case StepAdd in SQ. Then $e = \overline{n_1} + \overline{n_2}$ and $e' = \overline{n_1 + n_2}$. Two typing rules conclude a sum, so inversion gives two sub-cases:

- *Via TypeAddNN.* Then $\tau = \text{nn}$ and $\overline{n_1} : \text{nn}$, $\overline{n_2} : \text{nn}$. Inverting again (the only rule typing a numeral nn is TypeNumNN): $n_1 \geq 0$ and $n_2 \geq 0$. So $n_1 + n_2 \geq 0$, and by TypeNumNN, $e' : \text{nn}$.
- *Via TypeAddAny.* Then $\tau = \text{any}$. By TypeNum, $e' : \text{any}$.

□

Your turn. In EZ, only *one* rule concludes a sum, a difference, or a quotient, so StepAdd, StepSub, and StepDiv each have one case. Ask yourself what τ inversion forces, and which base typing rule retypes the resulting numeral. The Bonus (StepMul) is the two-rule situation shown above.

B4. Finding the type system’s blind spot (technique for Task 9)

Recipe. Find a rule that *throws away information* (its conclusion is weaker than what’s actually true for some inputs). Build an expression whose runtime value is fine, but only because of the information that got thrown away. Then propose extra information to track, and explain why the fix still blocks the bad programs.

Worked. In SQ, consider $\text{sqrt}(\overline{-2} * \overline{-3})$. At runtime, $(-2)(-3) = 6 \geq 0$, so it steps to $\overline{2}$ with no error. But $\overline{-2}$ and $\overline{-3}$ can only be typed any , so the product is any and TypeSqrt rejects it. The information “both factors are negative” was never recorded.

Fix: add a type np (“non-positive”) with

$$\frac{n \leq 0}{\overline{n} : \text{np}} (\text{TYPE_NUMNP}) \quad \frac{e_1 : \text{np} \quad e_2 : \text{np}}{e_1 * e_2 : \text{nn}} (\text{TYPE_MULNP})$$

Still safe: each new rule is backed by a true arithmetic fact (a product of two non-positives is ≥ 0), so nothing that evaluates to a negative can be typed nn . $\text{sqrt} \overline{-4}$ is still rejected.

Your turn: Task 9. Look at which EZ rules only ever conclude mz . The assignment says your fix doesn’t need to be complete or practical. One or two sentences of justification is fine.

Part C: Understanding the ideas

This part explains *why* the proof techniques in Parts A and B work. If a step in a proof feels like magic, the answer is probably here.

C1. What an inference rule actually says

A rule

$$\frac{e_1 : \text{int} \quad e_2 : \text{int}}{e_1 + e_2 : \text{int}} \text{ (TYPEADD)}$$

reads: *if* everything above the line holds, *then* the thing below holds. e_1, e_2 are placeholders, so the rule is a template: you get an *instance* by plugging in actual expressions.

A judgment “holds” **exactly when there is a derivation tree for it**, made of rule instances. Nothing else makes it true. That’s why

$$\bar{1} + \text{“a”} : \text{int}$$

does *not* hold. The only rule concluding a type for a $+$ is TypeAdd. Using it would need $\text{“a”} : \text{int}$, and the only rule that types a string literal (TypeString) concludes **string**. So no tree exists.

C2. Inversion: reading rules upside down

Rules normally go top to bottom (premises, so conclusion). Inversion goes the other way:

If a judgment holds, then *some* rule was used at the bottom of its tree. Look at every rule whose conclusion could match. Its premises must hold.

- **One matching rule:** you get its premises for free. Example: if $(e_1, e_2) : \tau$, only TypePair can conclude a pair’s type, so $\tau = \tau_1 \times \tau_2$ with $e_1 : \tau_1$ and $e_2 : \tau_2$.
- **Several matching rules:** you get one case per rule. This is what happened in B3 with TypeAddNN and TypeAddAny.
- **No matching rule:** the case is impossible, so you can skip it. Example: in the class’s Canonical Forms proof for **int**, the ValString case “doesn’t apply”, because a string literal can’t have type **int**.

How to spot it: whenever a proof says “we know e has *some* type, and e has a specific shape”, and then needs to know more, it’s inversion.

C3. Induction on derivations

This is ordinary induction, but on the size of a tree, not on a number.

- **Base cases:** axioms (rules with no premises of the judgment you’re inducting on). No IH is available.
- **Inductive cases:** rules with premises of that judgment. Each such premise is a *smaller* tree, so you get the IH for it.

This explains which cases use the IH:

Theorem	Induct on	IH available in
Preservation	$e \mapsto e'$	search rules (they have a $\mapsto$ premise)
Progress	$e : \tau$	rules like TypeAdd, TypeFst (they have a $:$ premise)

In Preservation, StepAdd and StepFst have no $\mapsto$ premise, so there's no IH, and you use inversion alone. In Progress, TypeNum has no premises, so it's a one-liner.

A common slip: the IH is only for the sub-derivation, with *its* type. In the TypeFst case of Progress, the IH is about $e_0 : \tau \times \tau_2$, not about e .

C4. Why Progress + Preservation = “can’t go wrong”

An expression is **stuck** if it is not a value and no step rule applies. Examples: $\bar{1} + \text{“a”}$, $\text{fst } \bar{3}$, $\text{sqrt } \bar{-4}$, $\bar{1}/\bar{0}$. A stuck expression is a run-time error.

Now chain the two theorems. Start with $e_0 : \tau$.

$$e_0 : \tau \xrightarrow{\text{Progress}} e_0 \text{ is a value, or } e_0 \mapsto e_1 \xrightarrow{\text{Preservation}} e_1 : \tau \xrightarrow{\text{Progress}} \dots$$

Every expression along the way is well-typed, and Progress says no well-typed expression is stuck. So evaluation either reaches a value or runs forever, but it never gets stuck.

- **Progress** checks the *dynamics*: there are enough step rules to handle every well-typed expression.
- **Preservation** checks the *typing rules*: they never promise something that evaluation doesn't deliver.

C5. Breaking things on purpose

The fastest way to understand a theorem is to see it fail.

(a) A missing step rule breaks Progress. Delete StepSearchPairR. Then $(\bar{3}, \bar{1} + \bar{2}) : \text{int} \times \text{int}$ is well-typed. It's not a value, since $\bar{1} + \bar{2}$ isn't. And no rule steps it: StepSearchPairL needs $\bar{3}$ to step, which it can't. So it's stuck. In the proof, the TypePair case breaks in the sub-case “ e_1 is a value and e_2 steps”: there's no rule left to cite.

(b) A dishonest typing rule breaks Preservation. Add a bogus rule to SQ:

$$\frac{e_1 : \text{nn} \quad e_2 : \text{nn}}{e_1 - e_2 : \text{nn}} \text{ (TYPESUBBAD)}$$

Now $\text{sqrt } (\bar{0} - \bar{1}) : \text{nn}$ type checks. But

$$\text{sqrt } (\bar{0} - \bar{1}) \mapsto \text{sqrt } \bar{-1}$$

(StepSearchSqrt $\leftarrow$ StepSub), which is stuck. Preservation failed at the StepSub step: $\bar{0} - \bar{1} : \text{nn}$, but $\bar{-1} : \text{nn}$ has no derivation (TypeNumNN needs $-1 \geq 0$). In the proof, the StepSub case would need $n_1 - n_2 \geq 0$, and nothing gives you that.

(c) **A too-strict typing rule breaks nothing.** Delete `TypeAddAny` from `SQ`. The system is still safe, but now $\overline{-1} + \overline{2}$ has no type, even though it runs fine. Type systems are *conservative*: they may reject good programs, but a safe one never accepts a bad one. Task 9 is about exactly this trade-off.

Part D: Practice problems

Solutions are at the end of this document. Try each one first. They use the pair language from Part A and the systems from Part B.

P1 (dynamic rules and proofs). Add `snd` e to the pair language.

1. Write its reduction rule, search rule, and typing rule.
2. Prove the Preservation case for your reduction rule.
3. Prove the Progress case for your typing rule.

P2 (two arguments). Add `max`(e_1, e_2) on integers. It evaluates e_1 , then e_2 , then returns the larger number. Write all its step rules and its typing rule, and prove the Progress case for the typing rule. How many IHs do you need?

P3 (evaluation and typing).

1. Evaluate `snd (fst (( $\bar{1}$ ,  $\bar{2} + \bar{3}$ ), “x”))` step by step, naming the rule stack at each step.
2. Give its typing derivation.

P4 (designing refinement rules). Language EV: types `ev` (“definitely even”) and `any`, expressions $e ::= \bar{n} \mid e + e \mid e * e \mid \text{half } e$, with

$$\frac{n \text{ even}}{\text{half } \bar{n} \mapsto \overline{n/2}} \text{ (STEPHALF)} \quad \frac{}{\bar{n} : \text{any}} \text{ (TYPENUM)} \quad \frac{n \text{ even}}{\bar{n} : \text{ev}} \text{ (TYPENUMEV)}$$

and $e : \text{ev}$ implies $e : \text{any}$. Design the rules:

1. $e_1 : \text{ev}, e_2 : \text{ev} \Rightarrow e_1 + e_2 : ?$ and $e_1 : \text{any}, e_2 : \text{any} \Rightarrow e_1 + e_2 : ?$
2. Multiplication: which operands must be `ev` for the product to be `ev`? Write the rules. How is this different from EZ’s `TypeMulNZ`?
3. $e : ? \Rightarrow \text{half } e : ?$

P5 (inversion with several rules). Using your P4 rules, prove the `StepMul` case of Preservation for EV. How many sub-cases does inversion give you?

P6 (blind spot). Find an expression in EV that never halves an odd number but doesn’t type check. Suggest a fix, and say why it is still safe. (Don’t use a sum. Look at `half` itself.)

P7 (which theorem breaks?). For each change, say whether Progress fails, Preservation fails, or neither (the system is just more restrictive). Give an example expression.

1. Remove `StepSearchFst` from the pair language.

2. Add the rule $\frac{e : \tau_1 \times \tau_2}{\text{fst } e : \tau_2} \text{ (TYPEFSTBAD)}$.

3. Remove `TypeMulAny` from EV.

LaTeX quick reference (this homework's macros.tex)

- `\Rule{Name}{prem1 \\\ prem2}{conclusion}`: a labeled rule. Use `{\strut}` for no premises.
- Nest a `\Rule` inside a premise to build derivation trees. Wrap big ones in `{\small ...}` or `\scriptsize`.
- Evaluation traces: an `array` with columns `{r l l}` (`\mapsto`, expression, rule names).
- Overlined numerals: `\overline{3}`.

Final checklist

- Task 1: 5 rules, exactly one search rule, no rule evaluates an un-chosen branch.
- Task 2: every step names its rule stack. Stop at a value.
- Task 3: every leaf is an axiom. Both branches get the same type.
- Tasks 4/6/8: say what you're inducting on. Each case names the rule it inverts on and the rule it concludes with.
- Task 8 Progress (TypeDiv): the $n_2 \neq 0$ side condition is justified by Canonical Forms, not assumed.
- Task 9: an expression, why it fails to type check, a fix, and why the fix is still safe.
- Task 10: don't forget it!

Solutions to Part D

P1.

$$\frac{e_1 \text{ val} \quad e_2 \text{ val}}{\text{snd } (e_1, e_2) \mapsto e_2} \text{ (STEPSND)} \quad \frac{e \mapsto e'}{\text{snd } e \mapsto \text{snd } e'} \text{ (STEPSEARCHSND)} \quad \frac{e : \tau_1 \times \tau_2}{\text{snd } e : \tau_2} \text{ (TYPESND)}$$

Preservation, Case StepSnd. Then $e = \text{snd } (e_1, e_2)$ and $e' = e_2$. By inversion on TypeSnd, $(e_1, e_2) : \tau_1 \times \tau$ for some τ_1 . By inversion on TypePair, $e_1 : \tau_1$ and $e_2 : \tau$. So $e' : \tau$.

Progress, Case TypeSnd. Then $e = \text{snd } e_0$ with $e_0 : \tau_1 \times \tau$. By the IH, $e_0 \text{ val}$ or $e_0 \mapsto e'_0$. If $e_0 \text{ val}$: by Canonical Forms, $e_0 = (e_1, e_2)$ with both values, so by StepSnd, $e \mapsto e_2$. If $e_0 \mapsto e'_0$: by StepSearchSnd, $e \mapsto \text{snd } e'_0$.

P2.

$$\frac{}{\text{max}(\overline{n_1}, \overline{n_2}) \mapsto \text{max}(n_1, n_2)} \text{ (STEPMAX)} \quad \frac{e_1 : \text{int} \quad e_2 : \text{int}}{\text{max}(e_1, e_2) : \text{int}} \text{ (TYPEMAX)}$$

$$\frac{e_1 \mapsto e'_1}{\text{max}(e_1, e_2) \mapsto \text{max}(e'_1, e_2)} \text{ (STEPSEARCHMAXL)}$$

$$\frac{e_2 \mapsto e'_2}{\text{max}(\overline{n_1}, e_2) \mapsto \text{max}(\overline{n_1}, e'_2)} \text{ (STEPSEARCHMAXR)}$$

Progress, Case TypeMax. Two IHs, nested, exactly like TypeAdd. By the IH on e_1 : if $e_1 \mapsto e'_1$, use StepSearchMaxL. If $e_1 \text{ val}$, then $e_1 = \overline{n_1}$ by Canonical Forms, and by the IH on e_2 : if $e_2 \mapsto e'_2$, use StepSearchMaxR; if $e_2 \text{ val}$, then $e_2 = \overline{n_2}$, and StepMax applies.

P3.

$$\begin{array}{ll} \text{snd } (\text{fst } ((\overline{1}, \overline{2} + \overline{3}), \text{"x"})) & \\ \mapsto \text{snd } (\text{fst } ((\overline{1}, \overline{5}), \text{"x"})) & \text{SSnd} \leftarrow \text{SFst} \leftarrow \text{SPairL} \leftarrow \text{SPairR} \leftarrow \text{StepAdd} \\ \mapsto \text{snd } (\overline{1}, \overline{5}) & \text{SSnd} \leftarrow \text{StepFst} \\ \mapsto \overline{5} & \text{StepSnd} \end{array}$$

(S = StepSearch. In step 1, SPairR needs $\overline{1} \text{ val}$. In step 2, StepFst needs $(\overline{1}, \overline{5}) \text{ val}$ by ValPair, and "x" val.)

Typing derivation. It's too wide for one tree, so name the inner pair $p = (\overline{1}, \overline{2} + \overline{3})$ and build it in two pieces (a handy trick for your own write-up too). First, the tree $\mathcal{D}$ for p :

$$\mathcal{D} = \frac{\frac{\overline{1} : \text{int}}{} \text{ (TN)} \quad \frac{\frac{\overline{2} : \text{int}}{} \text{ (TN)} \quad \frac{\overline{3} : \text{int}}{} \text{ (TN)}}{\overline{2} + \overline{3} : \text{int}} \text{ (TA)}}{\overline{1} : \text{int} \quad \overline{2} + \overline{3} : \text{int}} \text{ (TPAIR)}$$

Then the whole expression, with $\mathcal{D}$ plugged in as a premise:

$$\frac{\frac{\frac{\mathcal{D}}{} \text{ (TS)} \quad \text{"x"} : \text{string}}{(p, \text{"x"}) : (\text{int} \times \text{int}) \times \text{string}} \text{ (TPAIR)}}{\text{fst } (p, \text{"x"}) : \text{int} \times \text{int}} \text{ (TFST)}$$

$$\frac{}{\text{snd } (\text{fst } (p, \text{"x"})) : \text{int}} \text{ (TSND)}$$

P4.

1. $\text{ev} + \text{ev} : \text{ev}$ (even plus even is always even). $\text{any} + \text{any} : \text{any}$ (counterexample: $1 + 2 = 3$).
2. *One* even factor is enough, so there are three rules:

$$\frac{e_1 : \text{ev} \quad e_2 : \text{any}}{e_1 * e_2 : \text{ev}} \text{ (TYPEMULEVL)} \quad \frac{e_1 : \text{any} \quad e_2 : \text{ev}}{e_1 * e_2 : \text{ev}} \text{ (TYPEMULEVR)}$$

$$\frac{e_1 : \text{any} \quad e_2 : \text{any}}{e_1 * e_2 : \text{any}} \text{ (TYPEMULANY)}$$

In EZ you need *both* factors nonzero, because one zero factor makes the product zero. The arithmetic fact decides the shape of the rules.

3. The premise must be $e : \text{ev}$, to match StepHalf's side condition. The result is only any (counterexample: $\text{half } \bar{2} \mapsto \bar{1}$).

P5. $e = \overline{n_1} * \overline{n_2}$, $e' = \overline{n_1 n_2}$. Three typing rules conclude a product, so there are three sub-cases.

- *Via TypeMulEvL.* $\tau = \text{ev}$ and $\overline{n_1} : \text{ev}$. Inverting on TypeNumEv (the only rule giving a numeral type ev), n_1 is even. So $n_1 n_2$ is even, and by TypeNumEv, $e' : \text{ev}$.
- *Via TypeMulEvR.* Symmetric, with n_2 even.
- *Via TypeMulAny.* $\tau = \text{any}$. By TypeNum, $e' : \text{any}$.

P6. $\text{half} (\text{half } \bar{4})$ runs fine: $\mapsto \text{half } \bar{2} \mapsto \bar{1}$. But $\text{half } \bar{4}$ only gets type any , so the outer half rejects it. *Fix:* add a type m4 ("multiple of 4"), with $e : \text{m4}$ implying $e : \text{ev}$, and

$$\frac{4 \text{ divides } n}{\overline{n} : \text{m4}} \text{ (TYPENUMM4)} \quad \frac{e : \text{m4}}{\text{half } e : \text{ev}} \text{ (TYPEHALFM4)}$$

Still safe: if 4 divides n , then $n/2$ is even, so the new rule never promises too much.

P7.

1. **Progress fails.** $\text{fst} (\bar{1} + \bar{2}, \bar{3}) : \text{int}$, but it's not a value, StepFst needs a value pair, and there's no rule left to step inside fst . It's stuck.
2. **Preservation fails.** $\text{fst} (\bar{1}, \text{"a"}) : \text{string}$ by TypeFstBad, but it steps to $\bar{1}$, which isn't a string. Worse, $\text{fst} (\bar{1}, \text{"a"})^{\text{"b"}}$ type checks and gets stuck at $\bar{1}^{\text{"b"}}$.
3. **Neither.** The system is still safe but more restrictive: $\bar{1} * \bar{3}$ no longer has a type. (Careful: deleting a rule *can* break Preservation in general, if some step produces an expression only that rule could type. Here nothing does.)